Title: Shell Scripting Using Control & Looping Statements

1. AIM

To study and implement shell scripting using Bash programming language. To develop shell scripts using conditional decision-making statements such as if, if-else and case, evaluation operators, and looping statements such as for, while and until to perform automated tasks in Linux.

2. THEORY

2.1 Introduction to Shell Scripting

Shell scripting is a method of writing a sequence of Linux commands in a file to automate repetitive tasks. A shell script is executed by a command-line interpreter called a shell.

Bash (Bourne Again Shell) is one of the most commonly used shells in Linux operating systems.

Shell scripting helps users to:

- Automate repetitive system tasks.
- Perform mathematical calculations.
- Execute commands conditionally.
- Process files and directories.
- Implement loops and functions.
- Simplify system administration.

A shell script generally uses the `.sh` extension.

2.2 Bash Script Structure

A Bash script starts with a shebang line that specifies the interpreter.

```
None
#!/bin/bash
```

```
echo "Hello World"
```

Here:

- `#!/bin/bash` specifies the Bash interpreter.
- `echo` displays output on the terminal.
- Comments begin with `#`.

2.3 Variables in Shell Scripting

Variables are used to store values that can be accessed and modified during script execution.

Example:

```
None
name="Ravi"

age=21



echo "Name: $name"

echo "Age: $age"
```

The `$` symbol is used to access the value of a variable.

2.4 User Input in Shell Scripting

The `read` command accepts input from the user through the keyboard.

Example:

```
None
echo "Enter your name:"

read name
```

```
echo "Welcome $name"
```

2.5 Decision-Making Statements

Decision-making statements allow a shell script to execute different instructions depending on specified conditions.

A. if Statement

The if statement executes a block of commands when a condition is true.

Syntax:

```
None
if [ condition ]

then

    statements

fi
```

B. if-else Statement

The if-else statement executes one block when the condition is true and another block when it is false.

Syntax:

```
None
if [ condition ]

then

    statements

else

    statements
```

```
fi
```

C. Nested if-else Statement

A nested if-else statement contains one if-else statement inside another. It is used when multiple conditions must be checked.

D. case Statement

The case statement is used when a program needs to select one option from multiple choices.

Syntax:

```
None
case variable in

    pattern1)

        statements

        ;;

    pattern2)

        statements

        ;;

    *)

        default_statements

        ;;

esac
```

2.6 Evaluation Operators

Evaluation operators are used to compare values and perform logical operations.

Numeric Comparison Operators

Operator	Meaning
`-eq`	Equal to
`-ne`	Not equal to
`-gt`	Greater than
`-lt`	Less than
`-ge`	Greater than or equal to
`-le`	Less than or equal to

Logical Operators

Operator	Meaning
`&&`	Logical AND
`||`	Logical OR
`!`	Logical NOT

2.7 Looping Statements

Loops are used to execute a set of commands repeatedly until a specified condition is satisfied.

A. for Loop

The for loop executes commands for each value in a specified list or range.

Syntax:

```
None
for variable in list

do

    statements

done
```

B. while Loop

The while loop executes commands as long as the specified condition remains true.

Syntax:

```
None
while [ condition ]

do

    statements

done
```

C. until Loop

The until loop executes commands repeatedly until the specified condition becomes true.

Syntax:

```
None
until [ condition ]

do

    statements

done
```

2.8 Loop Control Statements

Loop control statements modify the normal execution of loops.

- `break`: Terminates the loop immediately.
- `continue`: Skips the current iteration and moves to the next iteration.

2.9 Functions in Shell Scripting

Functions are reusable blocks of commands that perform a particular task.

Syntax:

```
None
function_name()

{

    statements

}
```

Functions help in code reusability, modularity and easy maintenance.

3. CODE

The following programs demonstrate conditional statements, evaluation operators, looping statements and loop control in Bash.

Program 1: Check Whether a Number is Even or Odd Using if-else

```
None
#!/bin/bash


echo "Enter a number:"

read n


if [ $((n % 2)) -eq 0 ]
```

```
then
    echo "Number is Even"
else
    echo "Number is Odd"
fi
```

Program 2: Check Positive, Negative or Zero Using Nested if-else

```
None
#!/bin/bash

echo "Enter a number:"
read n

if [ $n -ge 0 ]
then
    if [ $n -eq 0 ]
    then
        echo "Number is Zero"
    else
        echo "Number is Positive"
    fi
else
```

```
    echo "Number is Negative"
fi
```

Program 3: Simple Calculator Using case Statement

```
None
#!/bin/bash

echo "Enter two numbers:"
read a b

echo "Select an operation:"
echo "1. Addition"
echo "2. Subtraction"
echo "3. Multiplication"
echo "4. Division"

read ch

case $ch in
    1)
        echo "Addition = $((a+b))"
        ;;
```

```
    2)
        echo "Subtraction = $((a-b))"
        ;;
    3)
        echo "Multiplication = $((a*b))"
        ;;
    4)
        if [ $b -ne 0 ]
        then
            echo "Division = $((a/b))"
        else
            echo "Division by zero is not allowed"
        fi
        ;;
    *)
        echo "Invalid choice"
        ;;
esac
```

Program 4: Sum of First N Natural Numbers Using while Loop

```
None
#!/bin/bash
```

```
echo "Enter a number:"
read n

i=1
sum=0

while [ $i -le $n ]
do
    sum=$((sum+i))
    i=$((i+1))
done

echo "Sum of first $n natural numbers = $sum"
```

Program 5: Print Numbers from 1 to 10 Using for Loop

```
None
#!/bin/bash

for i in {1..10}
do
    echo $i
```

```
done
```

Program 6: Print Numbers from 1 to 5 Using until Loop

```
None
#!/bin/bash


i=1


until [ $i -gt 5 ]
do
    echo $i
    i=$((i+1))
done
```

Program 7: Demonstrate break Statement

```
None
#!/bin/bash


for i in {1..10}
do
    if [ $i -eq 5 ]
    then
        break
```

```
    fi


    echo $i
done
```

Program 8: Demonstrate continue Statement

```
None
#!/bin/bash


for i in {1..10}
do
    if [ $i -eq 5 ]
    then
        continue
    fi


    echo $i
done
```

Program 9: Function to Perform Addition of Two Numbers

```
None
#!/bin/bash
```

```
addition()
{
    sum=$((a+b))
    echo "Addition = $sum"
}

echo "Enter two numbers:"
read a b

addition
```

Program 10: Function with Argument to Find Square of a Number

```
None
#!/bin/bash

square()
{
    echo "Square = $(($1 * $1))"
}

echo "Enter a number:"
read n
```

```
square $n
```

Program 11: Create and Write into a File

```
None
#!/bin/bash

echo "Enter file name:"
read file

touch "$file"

echo "Enter some text:"
read text

echo "$text" > "$file"

echo "File created successfully."

echo "File contents:"
cat "$file"
```

Program 12: Read a File and Check Whether It Exists

```
None
#!/bin/bash


echo "Enter file name:"
read file


if [ -f "$file" ]
then
    echo "File contents:"
    cat "$file"
else
    echo "File does not exist."
fi
```

4. EXECUTION INSTRUCTIONS

Step 1: Open Linux Terminal

Open Ubuntu or Kali Linux terminal using:

```
Ctrl + Alt + T
```

Step 2: Check Bash Availability

```
None
bash --version
```

Step 3: Create a Practical Directory

```
None
mkdir Shell_Scripting

cd Shell_Scripting
```

Step 4: Create a Shell Script

For example, to execute Program 1:

```
None
nano even_odd.sh
```

Write the Program 1 code in the editor.

Save the file using:

- Press `Ctrl + O`.
- Press Enter.
- Press `Ctrl + X`.

Step 5: Give Execute Permission

```
None
chmod +x even_odd.sh
```

Step 6: Execute the Script

```
None
./even_odd.sh
```

Step 7: Enter Input

Example:

```
None
Enter a number:
```

```
10

Number is Even
```

Step 8: Execute Other Programs

Repeat the same procedure for all remaining programs.

Use meaningful filenames:

Program	Filename
Even or Odd	`even_odd.sh`
Positive or Negative	`positive_negative.sh`
Calculator	`calculator.sh`
Sum using while	`sum.sh`
For loop	`for_loop.sh`
Until loop	`until_loop.sh`
Break	`break.sh`

Continue	`continue.sh`
Addition function	`addition.sh`
Square function	`square.sh`
File creation	`create_file.sh`
File reading	`read_file.sh`

5. RESULT

The Bash shell scripting programs were studied and implemented successfully.

The following operations were demonstrated:

1. Conditional decision-making using if-else statements.
2. Nested if-else statements for multiple conditions.
3. Multiple-choice operations using the case statement.
4. Repetitive execution using for, while and until loops.
5. Loop control using break and continue statements.
6. Function creation and function calling.
7. File creation, writing and reading using shell scripts.

6. CONCLUSION

In this practical, Bash shell scripting was studied and implemented using conditional statements, evaluation operators, looping statements and functions. Different programs were developed to perform mathematical calculations, check conditions, control loop execution and handle files. The practical demonstrated how shell scripting can automate repetitive Linux operations and improve the efficiency of system administration tasks.